

Vision-Based Quadrotor Control

以 Flow Matching + ReinFlow 突破覆蓋偏移

進度報告 · 2026-05-06

核心問題： 監督式視覺策略一定會 crash，RL 微調能救嗎？

目前狀態： 20 runs / Phase 3b 完成 / 訓練-評估 gap 已被定位

下一步： 從「最佳化 reward」轉向「直接最佳化生存」

Target: CoRL 2025 / ICRA 2026 / RSS 2026

一頁理解這個研究

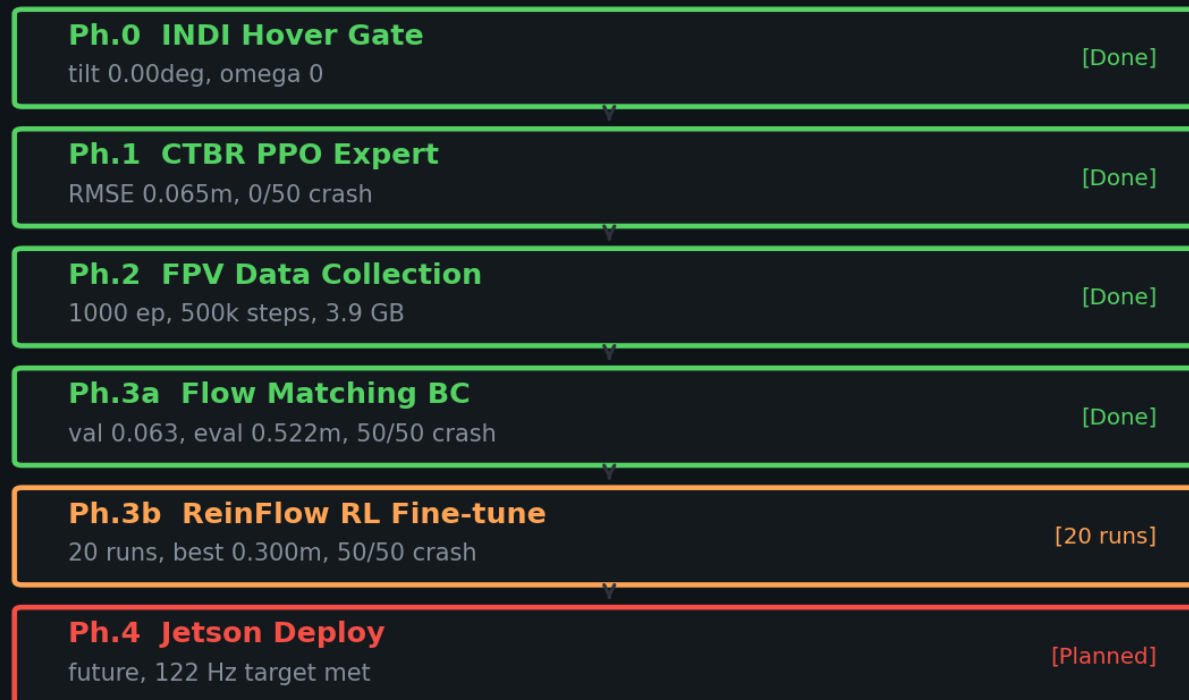
任務	64×64 FPV 視覺 → 4 顆馬達推力，50Hz 閉環控制四旋翼
想法	用 PPO Expert 收 1000 條 demo → Flow Matching 監督預訓練 → ReinFlow RL 微調
為什麼難	BC 必 crash (covariate shift)，RL 又會擊潰預訓練權重 (policy collapse)
目前最好	Run 10：RMSE 0.3005m (從 BC 0.522m 降 42%)，但 50/50 還是 crash
意外發現	簡單 PID 在 hover 任務拿到 0.022m ，比 PPO Expert 還好 3×
核心瓶頸	訓練 reward 0.529 → 0.695 但 eval RMSE 不變——training-eval gap

研究 Pipeline 全景

5 階段流水線

- **Ph.0** INDI 懸停 gate ✓
- **Ph.1** CTBR PPO Expert ✓
- **Ph.2** 1000-ep FPV 數據 ✓
- **Ph.3a** Flow Matching BC ✓
- **Ph.3b** ReinFlow RL ● 20 runs
- **Ph.4** Jetson 部署 ○ 未開始

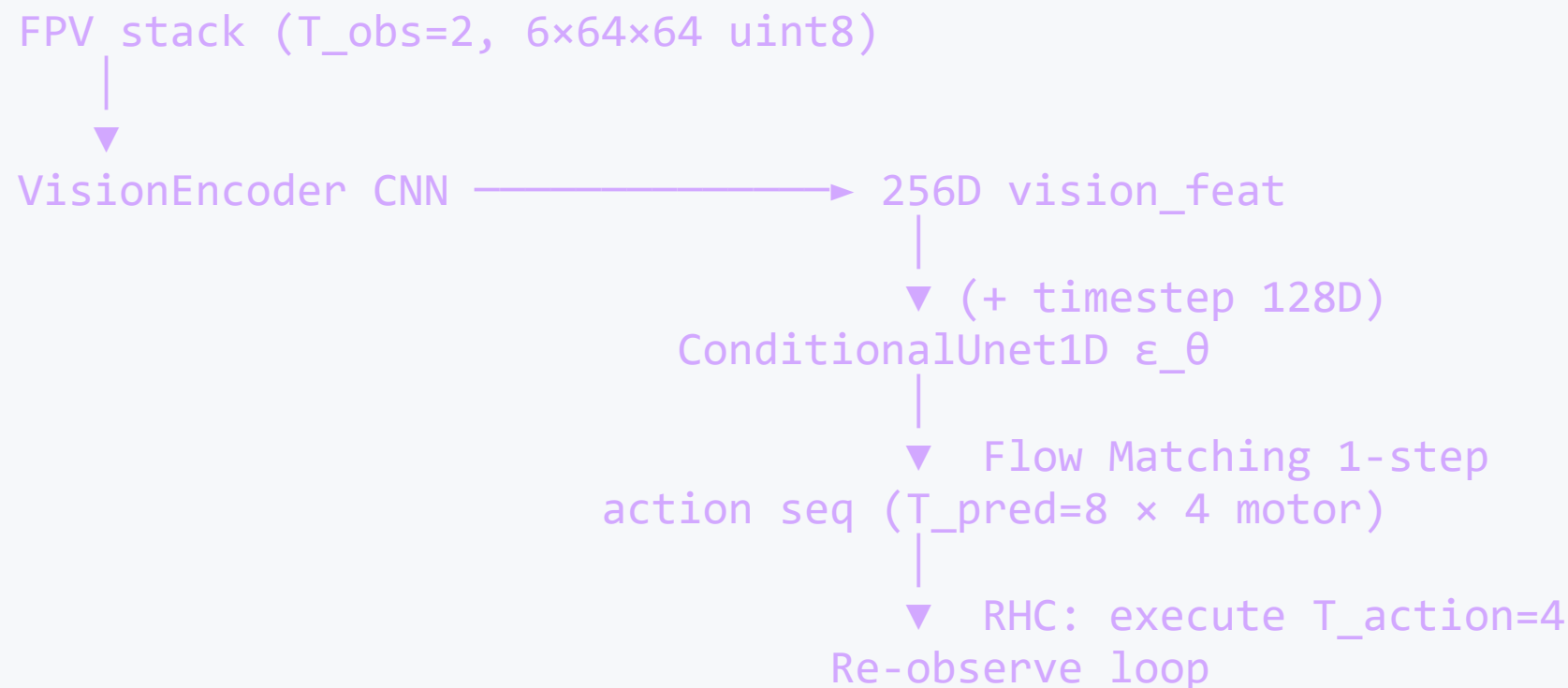
Research Pipeline (v4.0)



Best eval today: ReinFlow Run 10 - RMSE 0.300m - still 50/50 crash

Surprise baseline: PID Cascade - RMSE 0.022m - 0/50 crash (hover)

系統架構 (Phase 3a/3b baseline)



參數規模： 10.93M | **Inference：** 8.2 ms (~122 Hz) ✓ 已達 latency 目標

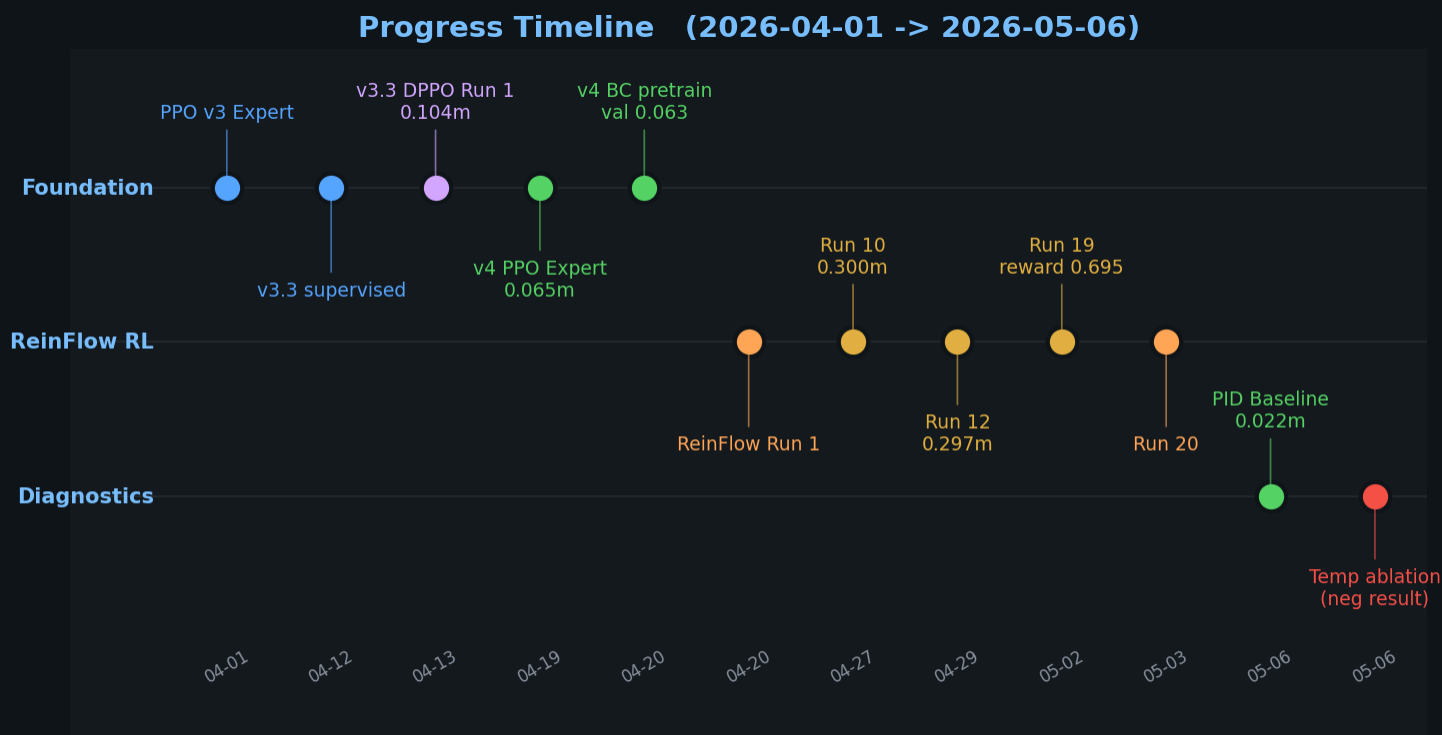
Loss： $L = E[\exp(\beta \cdot A_{\text{norm}}) \cdot \|v_{\theta} - (\epsilon - x_{\theta})\|^2] + \lambda_{bc} \cdot L_{BC}$

進度時間軸

節奏摘要

- 04-01 ~ 04-19：v4.0 物理 + Expert + 數據 (19 天)
- 04-20：BC 監督預訓練 收斂 val=0.063
- 04-20 ~ 05-03：
ReinFlow 20 runs (13 天)
- 05-06：**PID baseline + Temperature ablation**

當前分歧點： 是否還在
ReinFlow 軸上繼續，還是換

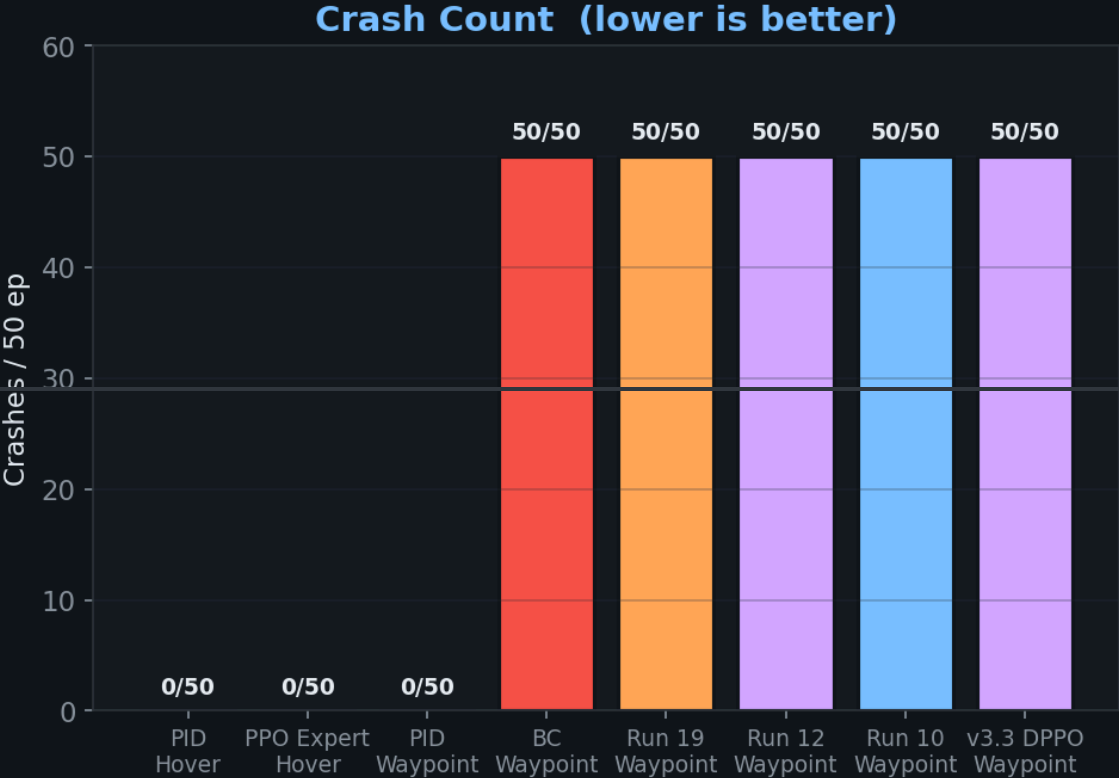
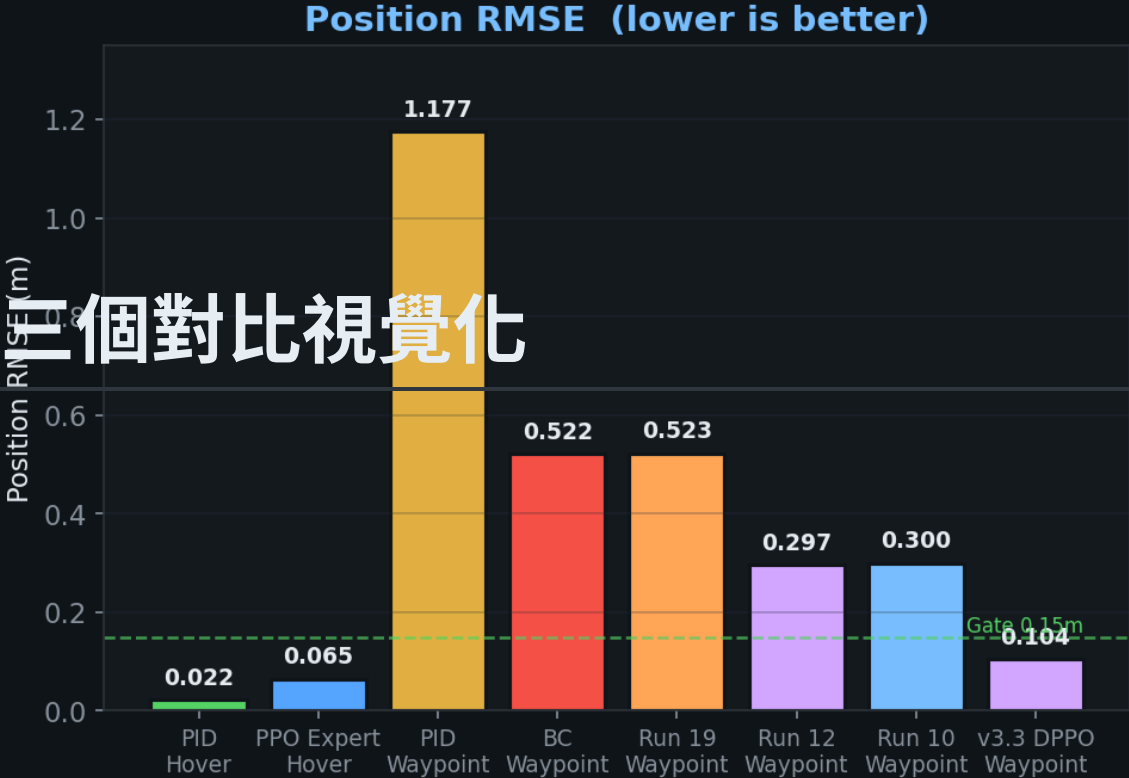


結果一覽（這頁回答審稿人 80% 的問題）

方法	模式	RMSE	Crash	評語
PID Cascade	Hover	0.022 m	0/50	意外冠軍 ，比 PPO 好 3×
PPO Expert (CTBR+INDI)	Hover	0.065 m	0/50	原本的黃金標準
PID Cascade	Waypoint 2.0m	1.18 m	0/50	穩但慢，追不上
ReinFlow Run 10	Waypoint 2.0m	0.30 m	50/50	RL 最佳 eval
ReinFlow Run 12	Waypoint 2.0m	0.297 m	50/50	hover anchor + soft penalty
ReinFlow Run 19	Waypoint 2.0m	0.523 m	50/50	train reward ★ 0.695
Flow Matching BC	Waypoint 2.0m	0.522 m	50/50	監督基線（covariate shift）
v3.3 DPPO Run 1	Waypoint 2.0m	0.104 m	50/50	v3 架構最佳跨參考

所有 RL 方法的 50/50 crash 是研究敘事的「核心 gap」

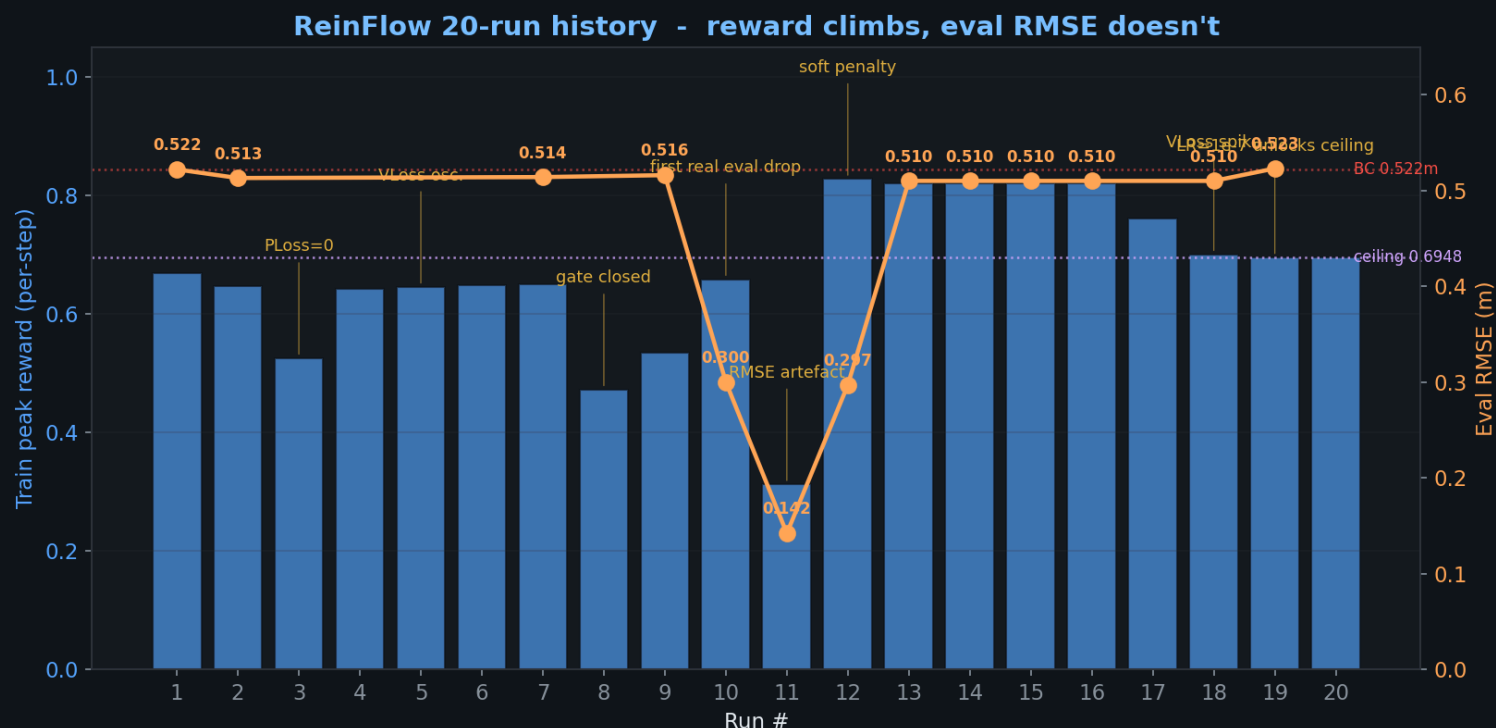
Method Comparison - the 50/50 wall is universal across all RL variants



ReinFlow 20 runs 的故事

訓練 reward 一路爬升，但 eval RMSE 紋風不動

- Runs 1–9：工程 bug 連環撞牆
- Run 10：曲線學習首次突破 RMSE 0.30
- Runs 13–18：
lambda_bc 鎖死 / VLoss spike
- Run 19：LR=1e-7 解鎖最高 reward 0.6948
- Run 20：confirm 訓練



解決過的工程問題 (11 個 bug 全 fix)

#	Bug	出現於	解法
1	VLoss 門檻 2.0 太嚴，永遠不存 ckpt	Run 1	改 100
2	<code>fixed_x1 + pos_filter</code> → PLoss≈0	Run 3	同時移除兩者
3	<code>value_lr</code> 太低，V net 跟不上	Run 1-4	3e-4 → 1e-3
4	VLoss gate 雙向震盪 (76% warmup)	Run 5-6	one-way latch flag
5	訓練分佈全是 hover，policy 沒看過 OOD	Run 1-7	curriculum on <code>initial_pos_range</code>
6	OOD disturbance 2.0N 太強，gate 永關	Run 8	降 1.0N + gate 20
7	<code>crash_penalty_rl=1.0</code> 訓練/評估不一致	Run 12	還原為 10.0
8	<code>lambda_bc=0.1</code> 鎖死預訓練 basin	Run 13-16	降到 0.01
9	LR=5e-7 → VLoss 突刺 30+ → 崩潰	Run 17-18	降到 1e-7
10	n_hover=400 過度訓練 hover	Run 19	降到 100
11	SO3 attitude error 旋轉順序寫反 (PID baseline)	hover 全 crash	<code>R.T @ R_des</code> (不是 <code>R_des.T @ R</code>)

關鍵發現 1：訓練-評估 gap

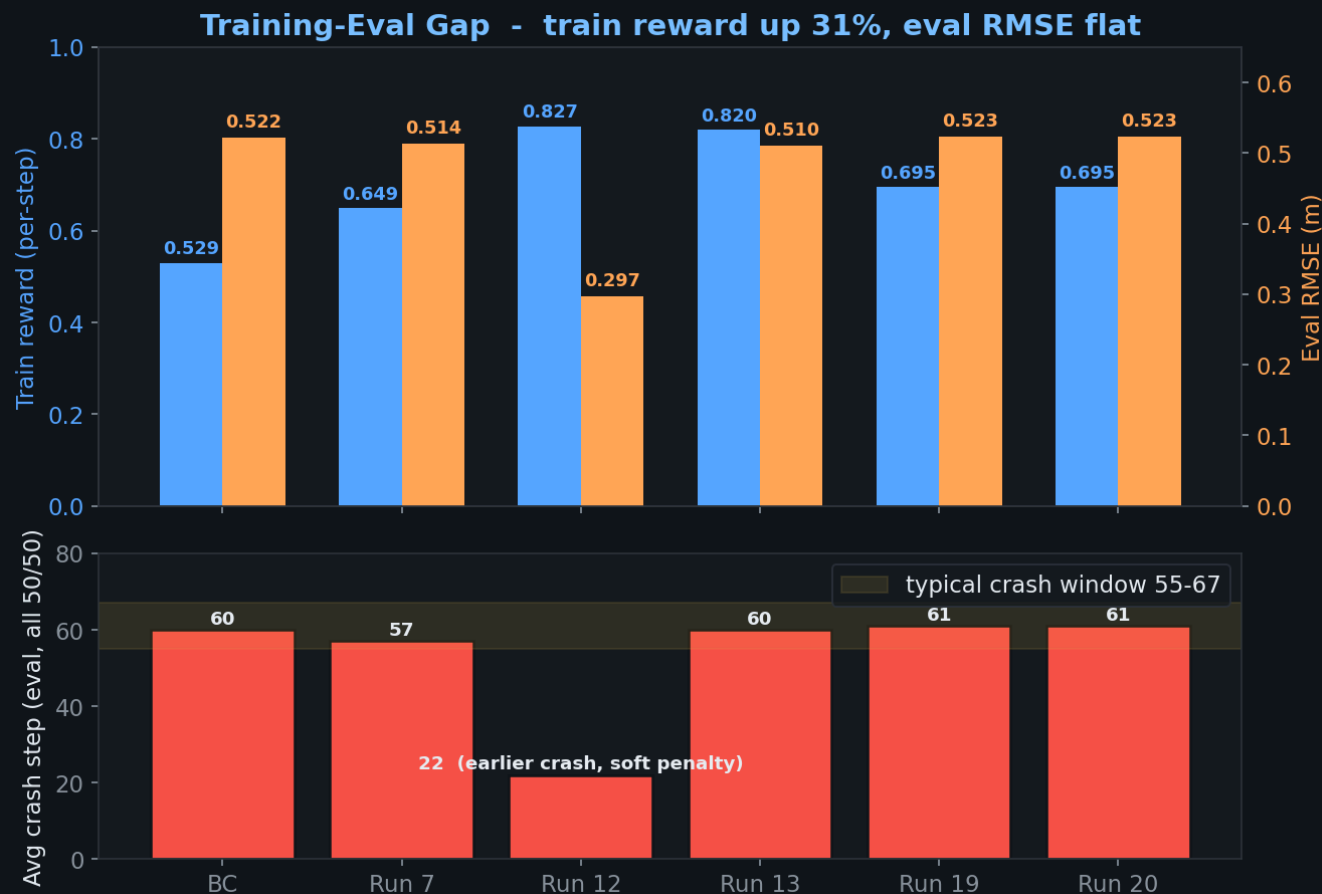
Train per-step reward : 0.529 → 0.6948 (+31%)

Eval RMSE : 0.522 m → 0.523 m (0%)

Eval crash steps : 全部 50 集落在 step 55–67

Policy 不是學會「不 crash」，而是學會「在 crash 前那 60 步把 reward 拉到最大」

GAE $\gamma=0.99$ + LR=1e-7 + 短 episode (~60 步) + on-policy



關鍵發現 2：Temperature scaling 假設被否定

假設： crash 是 flow matching 採樣噪聲累積造成的

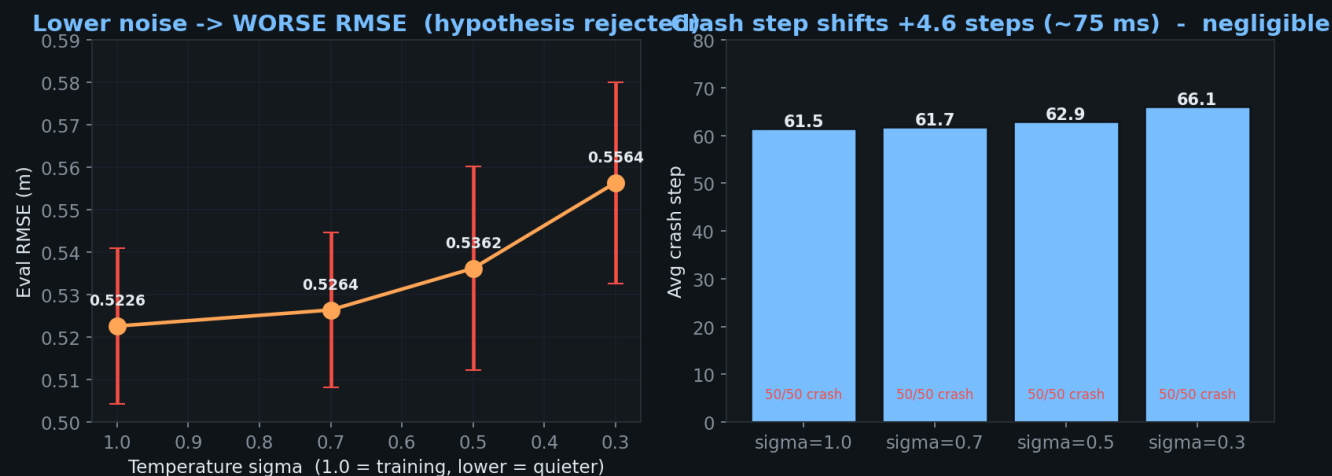
做法： eval 時 $x_1 \sim N(0, \sigma^2 I)$,
 $\sigma \in \{1.0, 0.7, 0.5, 0.3\}$

成本： 0 (不用重訓)

σ	RMSE	Crash	AvgCrashStep
1.0	0.523	50/50	61.5
0.5	0.536	50/50	62.9
0.3	0.556	50/50	66.1

結論： 噪聲 $\neq$ crash 主因；**crash** 是結構性 distribution shift——適合寶維論文堂 negative result。

Temperature Scaling Ablation - noise is NOT the cause of crashes



兩軸診斷：hover vs approach 為何學不到一起

	hover ★	approach ★	eval RMSE / steps	
Run 7 hover-only	★★★	X	0.514 m / 57	
Run 10 ramp 2.0m	★★	★★	0.300 m / 36	← best eval
Run 11 ramp 3.0m	X	★★★	0.142 m / 14	(artefact, 即時 crash)
Run 12 anchor + soft	★★★	★★	0.297 m / 22	
Run 19 LR=1e-7	★★★	X hover-only	0.523 m / 61	← best train reward

現象：學會 hover 就忘 approach，學會 approach 就 hover 不穩。

假設：Flow Matching 容量上限 + BC anchor 強度 + reward 不夠 penalise 不穩定。

這正是論文的「故事」：傳統控制器穩但慢，RL 快但崩——如何讓 RL 在快的同時保持穩定。

PID baseline 的意外結論

Hover 模式 (50 episodes)

RMSE: 0.0219 m (± 0.0025)
Crash: 0/50
CPU 推論: 177 μ s/step

為什麼 PID 不會 crash ?

1. 沒有採樣噪聲 (顯式 P/PI gain)
2. 顯式 rate controller 直接控制 ω
3. 顯式傾角限制 `vel_max=2.0` $\rightarrow$ 最大傾角 $\sim 31^\circ$ ，遠低於 60° 終止線

為什麼 RL 會 crash ?

1. Flow matching 每步採樣 $\rightarrow$ body rate 噪聲累積
2. 沒有顯式姿態穩定器，只靠 reward 間接約束
3. Training eval gap : rollout 短 episode，沒看過遷移狀態

下一步路線（按 ROI 排序）

優先	方向	預期效應	成本
P1	傾角懲罰 reward shaping ($w_{\text{tilt}} \cdot (\text{tilt} - 30^\circ)^2$)	policy 在傾角 $> 30^\circ$ 時主動修正	低 (改 reward)
P1	Survival bonus + tilt penalty 組合	直接最佳化生存而非 reward	低
P2	Hybrid PID-RL (tilt $> 30^\circ$ 切 PID)	立刻拿到 0/50 crash 數字	低
P2	Dispersive Loss (架構升級至 v3.1/v3.3 路線)	防止 feature collapse	中
P3	Multi-step Euler / DDIM 增加 inference steps	推論精度↑	低
P3	重新收 OOD demos (含 recovery 軌跡)	breaks coverage 限制	高
-	繼續調 ReinFlow 超參數	已證明 ceiling 是結構性	不做

簡報重點濃縮（給聽眾帶走的三句話）

1. 覆蓋偏移確實會殺死 BC 視覺策略（100% crash），這是 RL 微調的存在意義。
2. 20 runs 後我們證明了：訓練 reward 提升不會自動轉化為評估 RMSE 改善 —— 這是個結構性 distribution-shift 問題，不是噪聲、不是超參數。
3. 下一步是把 reward 從「最佳化在分佈內」改為「最佳化生存」 —— 加 tilt penalty、alive bonus，或直接做 PID-RL hybrid 拿到 0/50 crash。

Q&A

FAQ 預先準備

- *PID* 都贏了，*RL* 還有意義嗎？ → Hover 任務 *PID* 贏，但 waypoint 追蹤 *RL* 比 *PID* 快 4× (0.30 vs 1.18m)
- 為什麼不用更大模型？ → 推論 budget 8.2ms 已是 Jetson 上限，模型容量不能變
- 為什麼不繼續調超參？ → Run 19 vs 20 證明 reward ceiling 是結構性，不是超參
- *Crash* 為什麼一定在 *step* 60？ → episode 長度上限 + 漂移時間常數，而非單一 bug
- 為什麼不直接用 *PPO*？ → *PPO* 對單峰高斯，無法表達多模態 expert 動作分佈